简单线程池

1、threadpool.h

```
#include <pthread.h>
#include <assert.h>


//线程回调函数
struct job
{
    void* (*callback_function)(void *arg);    //线程回调函数
    void *arg;                                //回调函数参数
    struct job *next;
};


//线程池
struct threadpool
{
    int thread_num;                 //线程池中开启线程的个数
    int queue_max_num;              //队列中最大job的个数
    struct job *head;               //指向job的头指针
    pthread_t *pthreads;            //线程池中所有线程的pthread_t
    pthread_mutex_t mutex;          //互斥信号量
    pthread_cond_t queue_empty;     //队列为空的条件变量
    pthread_cond_t queue_not_empty; //队列不为空的条件变量
    pthread_cond_t queue_not_full;  //队列不为满的条件变量
    int queue_cur_num;              //队列当前的job个数
    int queue_close;                //队列是否已经关闭
    int pool_close;                 //线程池是否已经关闭
};


//================================================================================
//函数名:                   threadpool_init
//函数描述:                 初始化线程池
//输入:                     [in] thread_num     线程池开启的线程个数
//                          [in] queue_max_num  队列的最大job个数
//输出:                     无
//返回:                     成功:线程池地址 失败:NULL
//
```

```
//==========================================================================
struct threadpool* threadpool_init(int thread_num, int queue_max_num);


//==========================================================================
//函数名:                       threadpool_add_job
//函数描述:                     向线程池中添加任务
//输入:                         [in] pool                   线程池地址
//                              [in] callback_function      回调函数
//                              [in] arg                    回调函数参数
//输出:                         无
//返回:                         成功:0 失败:-1
//==========================================================================
int threadpool_add_job(struct threadpool *pool, void* (*callback_function)(void *arg)


//==========================================================================
//函数名:                       threadpool_destroy
//函数描述:                      销毁线程池
//输入:                         [in] pool                   线程池地址
//输出:                         无
//返回:                         成功:0 失败:-1
//==========================================================================
int threadpool_destroy(struct threadpool *pool);


//==========================================================================
//函数名:                       threadpool_function
//函数描述:                     线程池中线程函数
//输入:                         [in] arg                    线程池地址
//输出:                         无
//返回:                         无
//==========================================================================
void* threadpool_function(void* arg);
```

2、threadpool.c

```
#include "threadpool.h"


struct threadpool* threadpool_init(int thread_num, int queue_max_num)
```

```
{
    struct threadpool *pool = NULL;
    do
    {
        pool = (struct threadpool *)malloc(sizeof(struct threadpool));
        if (NULL == pool)
        {
            printf("failed to malloc threadpool!\n");
            break;
        }
        pool->thread_num = thread_num;//线程数
        pool->queue_max_num = queue_max_num;//队列大小
        pool->queue_cur_num = 0;//当前队列有多少
        pool->head = NULL;
        pool->tail = NULL;
        //初始化-互斥锁
        if (pthread_mutex_init(&(pool->mutex), NULL))
        {
            printf("failed to init mutex!\n");
            break;
        }
        //初始化-队列为空的条件变量
        if (pthread_cond_init(&(pool->queue_empty), NULL))
        {
            printf("failed to init queue_empty!\n");
            break;
        }
        //初始化-队列不为空的条件变量
        if (pthread_cond_init(&(pool->queue_not_empty), NULL))
        {
            printf("failed to init queue_not_empty!\n");
            break;
        }
        //初始化-队列不为满的条件变量
        if (pthread_cond_init(&(pool->queue_not_full), NULL))
        {
            printf("failed to init queue_not_full!\n");
            break;
        }
        //线程存活数
        pool->pthreads = malloc(sizeof(pthread_t) * thread_num);
        if (NULL == pool->pthreads)
        {
            printf("failed to malloc pthreads!\n");
```

```
            break;
        }
        pool->queue_close = 0;
        pool->pool_close = 0;
        int i;
        for (i = 0; i < pool->thread_num; ++i)
        {
            pthread_create(&(pool->pthreads[i]), NULL, threadpool_function, (void *)
        }

        return pool;
    } while (0);

    return NULL;
}


int threadpool_add_job(struct threadpool* pool, void* (*callback_function)(void *arg
{
    assert(pool != NULL);
    assert(callback_function != NULL);
    assert(arg != NULL);


    //加锁
    pthread_mutex_lock(&(pool->mutex));
    //队列满的时候
    while ((pool->queue_cur_num == pool->queue_max_num) && !(pool->queue_close || po
    {
        pthread_cond_wait(&(pool->queue_not_full), &(pool->mutex));   //队列满的时候就
    }
    if (pool->queue_close || pool->pool_close)    //队列关闭或者线程池关闭就退出
    {
        pthread_mutex_unlock(&(pool->mutex));
        return -1;
    }
    //工作结构体
    struct job *pjob =(struct job*) malloc(sizeof(struct job));
    if (NULL == pjob)
    {
        pthread_mutex_unlock(&(pool->mutex));
        return -1;
    }
```

```
    pjob->callback_function = callback_function;
    pjob->arg = arg;
    pjob->next = NULL;
    if (pool->head == NULL)
    {
        pool->head = pool->tail = pjob;
        pthread_cond_broadcast(&(pool->queue_not_empty));  //队列空的时候，有任务来时就
    }
    else
    {
        pool->tail->next = pjob;
        pool->tail = pjob;
    }
    pool->queue_cur_num++;
    pthread_mutex_unlock(&(pool->mutex));
    return 0;
}


void* threadpool_function(void* arg)
{
    struct threadpool *pool = (struct threadpool*)arg;
    struct job *pjob = NULL;
    while (1)  //死循环
    {
        pthread_mutex_lock(&(pool->mutex));
        while ((pool->queue_cur_num == 0) && !pool->pool_close)   //队列为空时，就等待
        {
            pthread_cond_wait(&(pool->queue_not_empty), &(pool->mutex));
        }
        if (pool->pool_close)   //线程池关闭，线程就退出
        {
            pthread_mutex_unlock(&(pool->mutex));
            pthread_exit(NULL);
        }
        pool->queue_cur_num--;
        pjob = pool->head;
        if (pool->queue_cur_num == 0)
        {
            pool->head = pool->tail = NULL;
        }
        else
        {
            pool->head = pjob->next;
```

```
136         }
137         if (pool->queue_cur_num == 0)
138         {
139             pthread_cond_signal(&(pool->queue_empty));        //队列为空，就可以通知th
140         }
141         if (pool->queue_cur_num == pool->queue_max_num - 1)
142         {
143             pthread_cond_broadcast(&(pool->queue_not_full));  //队列非满，就可以通知th
144         }
145         pthread_mutex_unlock(&(pool->mutex));
146 
147         (*(pjob->callback_function))(pjob->arg);   //线程真正要做的工作，回调函数的调用
148         free(pjob);
149         pjob = NULL;
150     }
151 }
152 int threadpool_destroy(struct threadpool *pool)
153 {
154     assert(pool != NULL);
155     pthread_mutex_lock(&(pool->mutex));
156     if (pool->queue_close || pool->pool_close)   //线程池已经退出了，就直接返回
157     {
158         pthread_mutex_unlock(&(pool->mutex));
159         return -1;
160     }
161 
162     pool->queue_close = 1;        //置队列关闭标志
163     while (pool->queue_cur_num != 0)
164     {
165         pthread_cond_wait(&(pool->queue_empty), &(pool->mutex));  //等待队列为空
166     }
167 
168     pool->pool_close = 1;      //置线程池关闭标志
169     pthread_mutex_unlock(&(pool->mutex));
170     pthread_cond_broadcast(&(pool->queue_not_empty));  //唤醒线程池中正在阻塞的线程
171     pthread_cond_broadcast(&(pool->queue_not_full));   //唤醒添加任务的threadpool_add_
172     int i;
173     for (i = 0; i < pool->thread_num; ++i)
174     {
175         pthread_join(pool->pthreads[i], NULL);    //等待线程池的所有线程执行完毕
176     }
177 
178     pthread_mutex_destroy(&(pool->mutex));          //清理资源
179     pthread_cond_destroy(&(pool->queue_empty));
```

```
    pthread_cond_destroy(&(pool->queue_not_empty));
    pthread_cond_destroy(&(pool->queue_not_full));
    free(pool->pthreads);
    struct job *p;
    while (pool->head != NULL)
    {
        p = pool->head;
        pool->head = p->next;
        free(p);
    }
    free(pool);
    return 0;
}
```

3、main.c

```
#include "threadpool.h"


void* work(void* arg)
{
    char *p = (char*) arg;
    printf("threadpool callback fuction : %s.\n", p);
    sleep(10);
}


int main(void)
{
    struct threadpool *pool = threadpool_init(10, 20);
    threadpool_add_job(pool, work, "1");
    threadpool_add_job(pool, work, "2");
    threadpool_add_job(pool, work, "3");
    threadpool_add_job(pool, work, "4");
    threadpool_add_job(pool, work, "5");
    threadpool_add_job(pool, work, "6");
    threadpool_add_job(pool, work, "7");
    threadpool_add_job(pool, work, "8");
    threadpool_add_job(pool, work, "9");
    threadpool_add_job(pool, work, "10");
```

```
    threadpool_add_job(pool, work, "11");
    threadpool_add_job(pool, work, "12");
    threadpool_add_job(pool, work, "13");
    threadpool_add_job(pool, work, "14");
    threadpool_add_job(pool, work, "15");
    threadpool_add_job(pool, work, "16");
    threadpool_add_job(pool, work, "17");
    threadpool_add_job(pool, work, "18");
    threadpool_add_job(pool, work, "19");
    threadpool_add_job(pool, work, "20");
    threadpool_add_job(pool, work, "21");
    threadpool_add_job(pool, work, "22");
    threadpool_add_job(pool, work, "23");
    threadpool_add_job(pool, work, "24");
    threadpool_add_job(pool, work, "25");
    threadpool_add_job(pool, work, "26");
    threadpool_add_job(pool, work, "27");
    threadpool_add_job(pool, work, "28");
    threadpool_add_job(pool, work, "29");
    threadpool_add_job(pool, work, "30");
    threadpool_add_job(pool, work, "31");
    threadpool_add_job(pool, work, "32");
    threadpool_add_job(pool, work, "33");
    threadpool_add_job(pool, work, "34");
    threadpool_add_job(pool, work, "35");
    threadpool_add_job(pool, work, "36");
    threadpool_add_job(pool, work, "37");
    threadpool_add_job(pool, work, "38");
    threadpool_add_job(pool, work, "39");
    threadpool_add_job(pool, work, "40");


    sleep(5);
    threadpool_destroy(pool);
    return 0;
}
```